

Lecture 1 - Software Dev Processes

Lecture 2 - Modern Software Engineering

Lecture 3 - Requirements

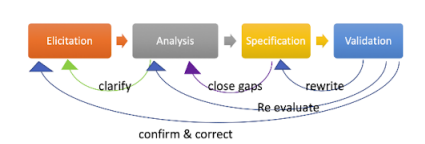
Requirements are a set of capabilities and constraints [IEEE]

1. A condition or capability needed by a user to solve a problem or achieve an objective
2. A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed documents
3. A documented representation fo a condition or capability as in (1) or (2)

Example of capability: system should allow a requester to specify pick-up and drop-off locations.

Example of constraint: pick-up and drop-off locations should be from a list of on-campus buildings / landmarks.

Requirements Development Phases



Elicitation: Discover requirements (through interview, workshop, document analysis, prototyping)

Analysis: Analyze, decompose, derive, understand, negotiate requirements, identify gaps

Specification: Written and illustrated requirements for comprehension, review, and use

Validation: Confirm correct set of requirements that will enable developers to build a solution

Sources of software requirements

- Documents
 - Descriptions of current or competing products
 - Standards or regulations
 - help desk problem reports
- Interviews
 - Focus groups
 - Use case workshops
 - Product champions as key customer rep
- Questionnaires and marketing surveys
 - Run a pilot first - difficult to ask clear questions
- Event-response tables
 - Identify external stimuli and describe system responses
- Observation - watch users do their jobs
 - Create workflow diagrams
 - "Day in the life" studies
 - Look at what information the user has when performing a particular task
- Prototyping
 - Useful for requirements, design, and implementation

FR vs NFR

FR describes something the system should do

- specify a behavior or function
- e.g. the passenger shall be able to print boarding passes for all flight segments for which she has checked in

NFR describes something not directly related to the functionality of the system

- May describe how well the system works
- Can include other attributes too (e.g. reliability, efficiency, usability)

- E.g. mean time between failure ≥ 9000 hrs

External vs Internal Quality Attributes

External:

- Observed when the software is executing
- Impacts user's experience of using the software
- Develops the user's perception of software quality
- E.g. Availability, performance, robustness, safety, security, reliability, deployability, compatibility, installability, usability, interoperability, integrity

Internal:

- Not directly observed when software is executing
- Perceived by the developers and maintainers
- Encompasses aspects of design that may impact external attributes
- E.g. Scalability, efficiency, modifiability, verifiability, portability, maintainability, reusability, testability

Sample problematic requirements

User service:

- F2 The platform should make sure that trusted users can perform important actions.
 - F2.1 Before a user edits a pickup place, withdraws an errand or marks delivery confirmed, the backend should verify that the user is allowed and return an error code otherwise.
 - F2.2 For every private screen / API call, the edge proxy shall decode the JWT and answer with an error code when teh credential is absent, expired or wrong.

Problems

- F2.1 has points related to supplier svc (editing pickup place) and order svc (withdrawing an errord or marking confirmed)
- Implementation details were included in the requirement (JWT)
- Explain trusted users
- Explain important actions

Supplier service:

- 3. Campus pickup point administration
 - 3.1 The system should display every usable vendor and handover point when a requester starts an errand
 - * 3.1.1 Each result should showw its label and type so users can choose it

Problems:

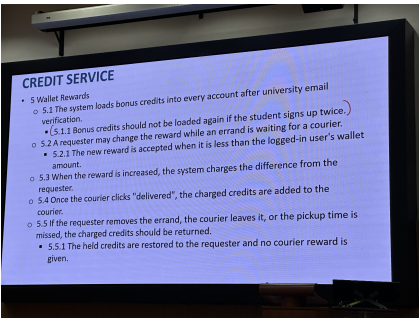
- Missing creating, deleting, editing
- Define "usable"

Supplier service:

- NFR3 Secure supplier administration
 - The system shall use proper permissions for the supplier directory
 - No confidential details should appear when a supplier request fails
 - Extra protction must be added to staff-only supplier URLs

Problems:

- "Proper permissions" is not testable
- Missing indexing



Problems:

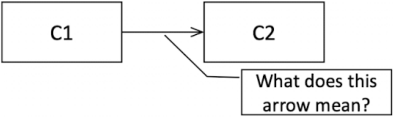
- 5.4: Missing necessary condition for credit transfer (requester has to complete)
- 5.1.1: User service should ensure that a student does not sign up twice

Lecture 4 - Software Architecture

The **Software Architecture** of a program or computing system is the **structure or structures of the system**, which comprise software components, the externally visible properties of those components, and the relationships between them. In short, the overall organization of the system and a high level design

Building blocks of architecture

- Component: Element that models an application-specific function/responsibility/requirement/task/process
- Connector:
- Configuration



Control flow

- Reasoning is on the computation order (computation in C2 depends on computation in C1)

Data flow

- Reasoning is on the data availability (C2 requires data from C1)

Can use solid / dashed lines to differentiate between the 2 flows and add a legend